

ERP Demo - API de catálogo e assistente local

Respostas técnicas e guia para executar a aplicação

Leonardo Briquezi

[GitHub: github.com/leobrqz](https://github.com/leobrqz)

[LinkedIn: linkedin.com/in/leonardobri](https://www.linkedin.com/in/leonardobri)

Esse PDF reúne as respostas às questões do enunciado, uma visão da implementação e as instruções essenciais de execução. A numeração original foi mantida.

Executar

No Windows, execute na raiz do projeto:

```
.\setup.ps1
```

O script prepara o `.env` quando necessário, constrói as imagens, inicia os serviços, aplica as migrações e executa o seed. As credenciais do administrador ficam em `ADMIN_USERNAME` e `ADMIN_PASSWORD` no `.env`.

No macOS/Linux, copie o arquivo de exemplo, configure `POSTGRES_PASSWORD`, `JWT_SECRET_KEY` e `ADMIN_PASSWORD`, e mantenha `DATABASE_URL` consistente com as variáveis do PostgreSQL:

```
cp .env.example .env
docker compose up --build -d
```

Para habilitar o modelo local no Windows:

```
.\setup.ps1 -EnableLocalAI
```

Esse comando habilita o perfil `ai` e baixa o MiniCPM5-2B Q4_K_M (~1,56 GB) para o volume `model_cache`. A inferência é local via `llama.cpp`; não requer token do Hugging Face nem usa API externa de LLM. O modo por regras continua disponível sem carregar o modelo.

Acessar a aplicação

Interface e documentação interativa:

```
http://localhost:8080
http://localhost:8080/docs
```

No Swagger, obtenha um JWT em `POST /api/v1/auth/token` com as credenciais do `.env` e informe o token em **Authorize**. A prontidão da API é exposta em `GET /health/ready`.

Rotas do desafio:

- Produtos: `POST/GET /api/v1/products` e `GET/PATCH/DELETE /api/v1/products/{id}`.
- Alertas e agregação: `GET /api/v1/alerts/low-stock` e `GET /api/v1/erp/snapshot`.
- Agente: `POST /api/v1/ai/ask` e `POST /api/v1/ai/ask/stream`. O streaming envia parágrafos; dados do catálogo vêm das ferramentas MCP.

Logs e encerramento:

```
docker compose logs -f api
docker compose down
```

Arquitetura e dados de demonstração

O Compose inicia frontend Nginx, API, PostgreSQL, Redis e worker RQ; API e worker compartilham a imagem do projeto. PostgreSQL persiste produtos, usuários e alertas. Redis atende cache e fila. O `model-server` com `llama.cpp` é opcional e executa MiniCPM5-2B localmente.

O seed cria um administrador local e quatro produtos fictícios: Café especial (R\$ 38,90; estoque 6), Caneca térmica (R\$ 59,90; estoque 24), Caderno pontilhado (R\$ 32,50; estoque 3) e Luminária de mesa (R\$ 119,00; estoque 12). As fontes de estoque, financeiro e clientes do endpoint agregado também são mocks, definidos em `app/services/erp_aggregate.py`.

Respostas da prova

Parte 1 - Arquitetura e organização

Questão 1 - Serviços e comunicação

Eu separaria os serviços por responsabilidade de negócio. O serviço de Produtos e Estoque seria dono do catálogo, saldo e movimentações; Pedidos coordenaria o ciclo do pedido; Financeiro continuaria dono de cobranças e recebimentos; Clientes continuaria dono dos dados cadastrais. Cada serviço manteria seu próprio banco e publicaria contratos estáveis, para que uma mudança interna não obrigasse os outros módulos a conhecerem suas tabelas.

Usaria REST para operações que precisam de resposta imediata, como validar se um cliente existe ou consultar o preço atual antes de aceitar um pedido. Usaria eventos e filas para efeitos que podem terminar depois, como notificar estoque baixo, atualizar relatórios e propagar uma venda confirmada. Eventos precisam de identificador, versão, data, produtor e chave de idempotência. Para não perder evento entre a transação do PostgreSQL e o broker, em produção eu adotaria transactional outbox e consumidores idempotentes.

PostgreSQL seria a fonte de verdade dos dados transacionais, com constraints para invariantes como preço e quantidade não negativos. Redis serviria para cache curto de leituras, rate limiting e filas quando o padrão operacional justificar. Não usaria cache ou lock distribuído como substituto de transação ou constraint no banco. Para operações de estoque concorrentes, a atualização seria atômica no PostgreSQL; um lock Redis só entraria se houvesse uma necessidade demonstrada e com expiração e recuperação bem definidas.

Um API Gateway como Kong centralizaria entrada, roteamento, TLS, autenticação de borda, rate limiting e políticas comuns. A autorização de negócio e as validações continuariam nos serviços, para não transformar o gateway em um serviço monolítico de regras.

Minha experiência prática com AWS é limitada. Conheço opções como ECS/Fargate para contêineres e RDS para PostgreSQL, mas não tenho profundidade para definir sozinho a arquitetura e a operação. Eu validaria mensageria, cache e observabilidade conforme os requisitos e com apoio de alguém experiente na plataforma.

Instrumentaria logs estruturados com `request_id/trace_id`, métricas Prometheus e tracing distribuído com OpenTelemetry, apresentados em Grafana. No ERP priorizaria latência p95/p99 e taxa de erro das rotas, disponibilidade e conexões do PostgreSQL, idade e tamanho das filas, retries e dead letters, falhas em integrações financeiras e divergências entre estoque reservado e disponível. Logs não devem registrar senhas, tokens nem dados pessoais desnecessários.

Questão 2 - Organização FastAPI

A estrutura desse projeto segue uma arquitetura em camadas, com dependências apontando para serviços de aplicação e repositórios, em vez de concentrar regras nas rotas:

<code>app/api</code> , <code>app/schemas</code>	HTTP e contratos Pydantic
<code>app/services</code>	casos de uso e regras
<code>app/repositories</code> , <code>models</code>	consultas e persistência SQLAlchemy
<code>app/db</code> , <code>app/core</code>	infraestrutura, configuração e segurança
<code>app/ai</code> , <code>app/workers</code>	agente MCP e tarefas RQ
<code>migrations</code>	histórico versionado do schema

Rotas e schemas tratam HTTP; serviços coordenam regras; repositórios isolam a persistência.

Parte 2 - Assíncrono e concorrência

Questão 3 - asyncio, threading e multiprocessing

asyncio multiplexa operações de I/O cooperativo em uma thread. É uma boa opção quando há muitas esperas de rede ou banco e as bibliotecas usadas oferecem APIs async. No ERP, eu chamaria simultaneamente os serviços de estoque, financeiro e clientes, como no endpoint da Questão 4.

threading é útil para I/O bloqueante quando a biblioteca ainda não tem API assíncrona, ou para manter várias esperas de rede em andamento. No ERP, uma integração antiga de emissão de nota que só oferece cliente bloqueante poderia ser isolada em uma thread para não travar o loop async. O GIL limita ganho de threads para código Python puro que consome CPU.

multiprocessing executa trabalho CPU-bound em processos separados e pode usar vários núcleos, pagando o custo de serializar dados e criar processos. No ERP, seria apropriado para consolidar milhões de linhas de um CSV ou gerar um relatório PDF pesado. Esse trabalho também pode ficar em um worker para não ocupar o processo HTTP.

Parte 3 - API RESTful

Questão 4 - Três fontes em paralelo

O endpoint GET /api/v1/erp/snapshot usa asyncio.gather. Estoque, financeiro e cliente são funções mock com latências independentes. Cada chamada passa por timeout próprio e retry simples para timeout/falha transitória. As exceções são convertidas em status por fonte; uma fonte indisponível não remove os resultados das outras. A resposta marca degraded quando houve resultado parcial. Em produção, essas funções seriam clientes HTTP com timeout total, retry limitado com jitter, circuit breaker e métricas.

Questão 6 - Produtos e estoque

A API implementa criar, listar, ler, atualizar parcialmente e excluir produto. O schema Pydantic exige nome textual e preço/estoque não negativos. Escolhi SQLAlchemy 2 com Psycopg 3 síncrono pela API de transações e consultas madura e pelo reaproveitamento da mesma camada no worker RQ; as rotas síncronas do FastAPI rodam no thread pool. Reservei asyncio para o endpoint que realmente precisa fazer várias chamadas de I/O em paralelo. PostgreSQL mantém dados duráveis; JWT protege as rotas e o seed cria um usuário admin de demonstração. Listagens aceitam nome, preço mínimo/máximo, quantidade máxima de estoque, página e tamanho limitado.

GET /api/v1/products usa Redis com chaves que incluem um contador de versão e os filtros normalizados. Depois de uma escrita, o contador é incrementado: as novas leituras passam a uma nova versão de chave, e chaves antigas expiram em cinco minutos. Se Redis estiver indisponível, a listagem consulta PostgreSQL e a mutação do produto continua funcionando.

RQ usa o mesmo Redis para enfileirar refresh_low_stock_alert após criar produto ou mudar quantidade. O worker reavalia o saldo e abre, atualiza ou resolve o alerta persistido em PostgreSQL. A tarefa altera estado real do domínio, e GET /api/v1/alerts/low-stock consulta os alertas abertos. Se o Redis falhar entre o commit do produto e o enqueue, a falha fica registrada em log; em produção eu acrescentaria uma transactional outbox para garantir entrega eventual desse trabalho.

Parte 4 - Docker e entrega

Questão 7 - Docker Compose

O Dockerfile usa build multi-stage e executa a API sem root. O Compose inicia Nginx, API, PostgreSQL, Redis e worker; llama.cpp fica no perfil opcional ai. Nginx encaminha /api/ e mantém o streaming SSE sem buffering. A API aplica migrações e seed idempotente ao iniciar. Para repetir, use docker compose exec api alembic upgrade head ou docker compose exec api python -m app.seed. O .env local é ignorado pelo Git; .env.example contém valores de demonstração.

Parte 5 - Agente, LLM e MCP

Questão 8 - Consulta em linguagem natural

POST /api/v1/ai/ask aceita perguntas em português. Com AI_PROVIDER=local, MiniCPM5-2B interpreta o sentido do pedido, escolhe uma ferramenta MCP de leitura e usa seu resultado para responder. A ferramenta e o repositório validam os argumentos e limitam a consulta; o usuário não fornece SQL. Se a intenção estiver ambígua, o assistente pede esclarecimento. Se o modelo estiver indisponível, a aplicação informa a falha sem tentar interpretar o texto com uma lista de frases fixas.

O agente usa LangChain, ChatOpenAI apontado para a API compatível do servidor local llama.cpp e MCPAdapter para descobrir as ferramentas. Os pesos MiniCPM5-2B vêm do repositório oficial do OpenBMB no Hugging Face, em quantização Q4. O modelo tem 2,52B parâmetros e o arquivo quantizado ocupa cerca de 1,56 GB. Não há chamada a OpenAI, Anthropic nem inferência hospedada. O modo determinístico continua disponível ao configurar AI_PROVIDER=rules.

Para habilitar o modelo local:

1. Rode ./setup.ps1 -EnableLocalAI no PowerShell, ou configure AI_PROVIDER=local no .env.
2. O Compose inicia model-server, que baixa openbmb/MiniCPM5-2B-GGUF:Q4_K_M do Hugging Face e mantém o arquivo no volume model_cache.
3. A API conversa com o modelo em http://model-server:8080/v1; http://localhost:8081/health mostra quando o serviço está pronto.

Os pesos ficam no volume model_cache; o modelo público não exige token do Hugging Face. O timeout padrão é de 60 segundos e pode ser ajustado em LLM_TIMEOUT_SECONDS. Se a chamada falhar, a API retorna erro sem interpretar a pergunta por regras de texto.

Questão 9 - Plugabilidade, tools, MCP e guardrails

O servidor FastMCP em app/ai/mcp_server.py expõe find_low_stock_products, search_products e list_products. São ferramentas somente de leitura, limitadas e com argumentos tipados. LangChain MCPAdapter descobre essas ferramentas e as converte em ferramentas que create_agent pode usar. A resposta de contagem usa o total consultado no banco, mesmo quando a lista de itens é limitada. A implementação conecta o MCP em memória para manter a demonstração simples; em produção o servidor MCP poderia ficar ao lado dos microsserviços e chamar seus APIs/casos de uso, sem acessar diretamente bancos de outros domínios.

Para conectar um LLM diferente, eu manteria a escolha do provedor em configuração e seguiria o contrato de chat/tool calling do LangChain. O restante do caso de uso recebe ferramentas tipadas e não conhece detalhes do provedor. O modo por regras pode ser selecionado explicitamente para execução sem modelo.

O agente atual não oferece ferramentas de escrita. Se fossem adicionadas, exigiriam autorização de negócio no serviço, validação Pydantic, allowlist de ações e confirmação explícita para operações destrutivas. O resultado do modelo nunca concede permissão; cada chamada é validada com o usuário autenticado. Dados vindos de consultas são tratados como conteúdo, não como instruções.

Eu acompanharia latência, timeouts, falhas, tokens quando disponíveis, ferramentas inválidas e fallback. A inferência local elimina o custo por token de API, mas consome memória e CPU. Registraria métricas e IDs de correlação, sem guardar prompts ou respostas brutos por padrão. Cache só para consultas de leitura, com chave por usuário, filtros e versão dos dados.

O namespace langchain.mcp está em beta na documentação atual e pode mudar. Para reduzir impacto em atualizações, eu fixaria a versão e isolaria o adaptador; as ferramentas MCP continuariam atrás de contratos tipados.

Parte 6 - Perfil

Questão 10 - Desenvolver uma frente em Go

Eu começaria pelos requisitos e métricas atuais. Go pode ajudar na ingestão concorrente, mas só migraria após comparar memória, latência p95/p99, throughput e custo operacional com o serviço atual. Se Python async ou uma otimização do banco atingir as metas, não justificaria adicionar outra linguagem. Caso Go seja a melhor opção, faria uma migração gradual, com cancelamento de contexto, erros explícitos, testes e profiling.

Parte 7 - Portfólio

Questão 11 - Projeto representativo

Projeto que escolho destacar: [SmokeShopERP-DataAnalytics](#).

É uma aplicação desktop para acompanhar produtos, vendas, clientes e indicadores de uma tabacaria, feita com Python, PyQt6 e PostgreSQL; Matplotlib e NumPy geram gráficos e análises. Se retomasse o projeto, separaria melhor interface, casos de uso e persistência, adicionaria migrações e testes, e só criaria uma API se houvesse necessidade de múltiplos clientes ou acesso remoto.

Uso de IA e revisão

Codex CLI apoiou a pesquisa, a comparação de alternativas e a geração inicial de partes do código e das respostas. O agente usa MiniCPM5-2B localmente; a aplicação não envia consultas a um LLM hospedado. A preparação desse PDF também contou com IA por meio de um script Python.

Limitações da demonstração

- A agregação de estoque, financeiro e clientes usa mocks locais.
- O agente só consulta dados; a autenticação demonstra um único administrador de seed.
- O enqueue RQ é best-effort; uma transactional outbox não foi implementada.
- Esta versão não inclui uma suíte automatizada de testes.
- Repositório: [demo-erp](#).

Referências técnicas

[Documentação FastAPI](#) | [Documentação SQLAlchemy](#) | [Documentação Alembic](#) | [Documentação RQ](#) | [Integração LangChain com MCP](#) | [Model card MiniCPM5-2B](#) | [MiniCPM5-2B GGUF](#) | [llama.cpp server](#)